

Enterprise Knowledge Intelligence Platform with Hybrid Relational, Document, and Vector Data Architecture

*A Project Based Learning Report Submitted in partial fulfilment of the requirements for the
award of the degree*

of

Bachelor of Technology

in The Department of Artificial Intelligence and Data Science

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Submitted by

2510080006: M. Sathya Krishna

2510080001: Yashwanth

2510080005: Aneeq

2510080002: Abhinav

Under the guidance of

Anitha. P



Department of Artificial Intelligence and Data Science

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCT - 2026.

Abstract

Organisations keep their knowledge in policies, reports and manuals that are hard to find by keyword alone, and every result must respect who may read it. The Enterprise Knowledge Intelligence Platform (EIP) stores each kind of data in the database that suits it: PostgreSQL holds users, roles, permissions and document metadata in a normalised schema; MongoDB holds document text, chunks and flexible metadata; and Qdrant holds 384-dimensional embeddings of every chunk for search by meaning.

A Spring Boot REST API, secured with JSON Web Tokens and role-based access control, joins the three stores. Users upload PDF, Word, text or Markdown files; the server extracts the text, splits it into chunks, embeds them and writes all three databases, undoing earlier writes if a later one fails. Search fuses a keyword leg that tolerates typing mistakes with a semantic leg, and filters every result by one document-level permission rule.

The system passes 191 backend tests against containerised databases, 148 frontend tests, a 28-check smoke test and a 50-user load test (169 requests per second, no errors). On a 315-document corpus, semantic search found the remote-working policy for “working from home rules”, which shares no keyword with it, and real Word-generated PDF and DOCX files were extracted and made searchable in about two seconds. The platform runs on free hosting: Vercel, Render, Neon, MongoDB Atlas and Qdrant Cloud.

Keywords: polyglot persistence, PostgreSQL, MongoDB, Qdrant, vector search, text extraction, Spring Boot, JWT, role-based access control

List of Figures

Figure 1: Three-tier architecture of the platform	7
Figure 2: Entity-relationship diagram of the PostgreSQL schema (crow's-foot notation)	8
Figure 3: One demo document (id 44) as stored in PostgreSQL, MongoDB and Qdrant	9
Figure 4: Text extraction: checks, one extractor per format, and the error returned at each step	10
Figure 5: Upload pipeline across the three databases, with compensating actions on failure	10
Figure 6: Authorisation checks on every request, and the response when one fails	11
Figure 7: Hybrid search: two legs, one permission filter, weighted fusion	12
Figure 8: Dashboard with live counts from PostgreSQL and Qdrant (316 documents and 718 vectors, including the uploaded handbook).....	15
Figure 9: Hybrid search for “working from home rules”, matched by meaning	16
Figure 10: Upload form with a PDF chosen	16
Figure 11: An uploaded PDF in the viewer: MongoDB text and chunks, extractor pdffbox-3.0	17
Figure 12: User administration (USER_MANAGE only).....	18

List of Tables

Table 1: Project modules and their implementation	7
Table 2: PostgreSQL tables	8
Table 3: Roles and their permissions	11
Table 4: Main REST endpoints	11
Table 5: Test results.....	13
Table 6: Upload checks with real files.....	13
Table 7: Search results on the 315-document demo corpus.....	13
Table 8: What each demo user can see	14
Table 9: Performance measurements	14
Table 10: Defects found by testing, and their fixes	14

Table of Contents

1. Introduction	6
2. Methodology	7
a. Requirements and Architecture	7
b. Relational Design in PostgreSQL	8
c. Document Model in MongoDB and Vectors in Qdrant	9
d. Text Extraction for PDF, Word, Text and Markdown	9
e. Keeping Three Databases Consistent	10
f. API and Security	11
g. Hybrid Search	12
h. Frontend and Deployment	12
3. Experiments	12
4. Results	13
5. Conclusion and Future Work	18
References	18

Enterprise Knowledge Intelligence Platform with Hybrid Relational, Document, and Vector Data Architecture

1. INTRODUCTION

Every organisation produces documents its people later need to find: HR policies, finance procedures, engineering runbooks, legal templates and research notes. Usually they sit in shared drives, and finding one depends on remembering its exact words. An employee who asks “what are the rules for working from home?” finds nothing if the policy is titled *Hybrid and Remote Working Standard*. At the same time, salary guidelines, contracts and security procedures must reach only the right people; even a leaked title in a search result is a breach.

No single database serves all of this well. A relational database [2], [3] is the right home for users, roles, permissions and metadata, where constraints and transactions enforce the rules. Long, variable document text with flexible metadata suits a document database. Finding passages that *mean* something close to a question needs vector embeddings and approximate nearest-neighbour search [4], [5]. Using several databases, each for what it does best, is *polyglot persistence* [1]; its cost is keeping the databases consistent without a transaction that spans them.

Problem statement. Build an enterprise knowledge platform that stores, organises and retrieves documents using relational, document and vector databases together; accepts common office formats; finds documents by keyword, approximate spelling and meaning; never shows a user a document they may not read; and keeps the three databases consistent as documents are added and removed.

Approach. The platform gives each database one job and links them through a single key, the PostgreSQL document id. Uploaded files are turned into plain text on the server, split into overlapping chunks and embedded, so a policy can be found by what it says, not only by its title. All writes go through one service that can undo them, and all reads pass one permission rule, so correctness and security do not depend on each endpoint getting the details right.

Objectives.

- Design a normalised (3NF) PostgreSQL schema for identity, roles, permissions, metadata, versions, grants and search history, with constraints and indexes.
- Model document content in MongoDB with schema validation and indexes, linked to PostgreSQL by id.
- Extract text from PDF, Word (.docx), text and Markdown uploads safely.
- Store chunk embeddings in Qdrant and support semantic and hybrid search.
- Secure the REST API with JWT, BCrypt, role permissions and per-document grants, applied on every endpoint.
- Keep the three databases consistent on upload and delete, then containerise, test and deploy on free hosting.

Scope. The project is Project 1 of the course portfolio and covers its five modules: user and role management, the knowledge repository, document and semantic search, REST APIs with authentication, and deployment, testing and documentation. Evaluation uses a demo corpus of 315 English documents in six categories. Section 2 describes the design, Section 3 the experiments and Section 4 the results; Section 5 concludes.

2. METHODOLOGY

a. Requirements and Architecture

Employees search and read documents they may access; **managers** can also upload and share their own documents; **administrators** read everything and manage accounts. Table 1 maps the handbook’s modules to what was built, and Figure 1 shows the three tiers.

Table 1: Project modules and their implementation

Handbook module	Implementation
1. User and role management	Users, roles, permissions and junction tables; admin page to create users, change roles, disable accounts, reset passwords
2. Knowledge repository	Metadata and versions in PostgreSQL, text and chunks in MongoDB; PDF, DOCX, TXT and MD upload; paged, filtered repository; delete
3. Document and semantic search	Hybrid, semantic, keyword and fuzzy modes; MiniLM embeddings in Qdrant; per-user search history
4. REST API and authentication	Spring Boot REST API; JWT sign-in; BCrypt; permission-based access; per-document grants
5. Deployment, testing, documentation	Docker images and Compose; 191 backend and 148 frontend tests; smoke and load tests; live on free hosting

The React frontend is served by Vercel, which also forwards `/api` to the backend, so the browser talks to one origin. The Spring Boot backend authenticates each request, applies the business rules and talks to PostgreSQL through JPA, to MongoDB through Spring Data and to Qdrant over gRPC. The embedding model runs inside the backend through ONNX Runtime, so no model server is needed.

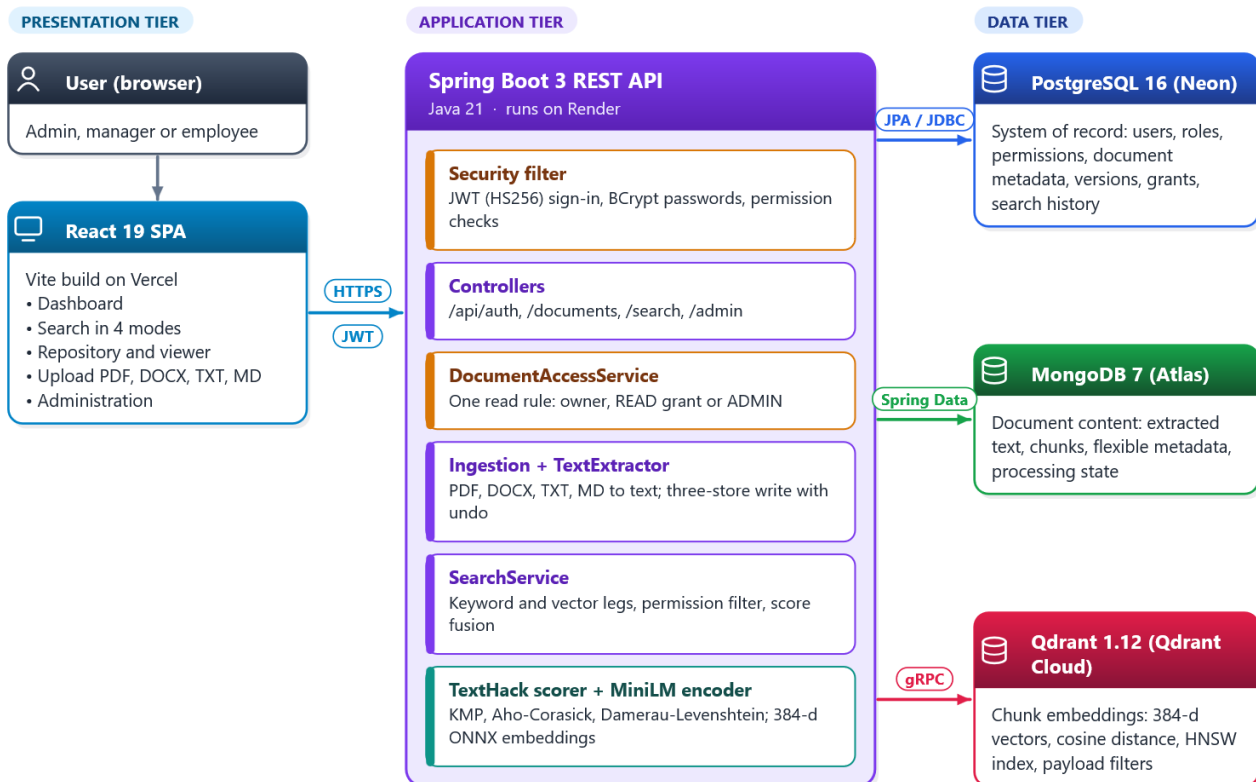


Figure 1: Three-tier architecture of the platform

b. *Relational Design in PostgreSQL*

PostgreSQL is the system of record: it decides who exists, what they may do and which documents exist. Its twelve tables (Figure 2, Table 2) resolve the two many-to-many relationships (users–roles, roles–permissions) through junction tables with composite keys, and give each document a category, an owner, versions, tags, grants and search history.

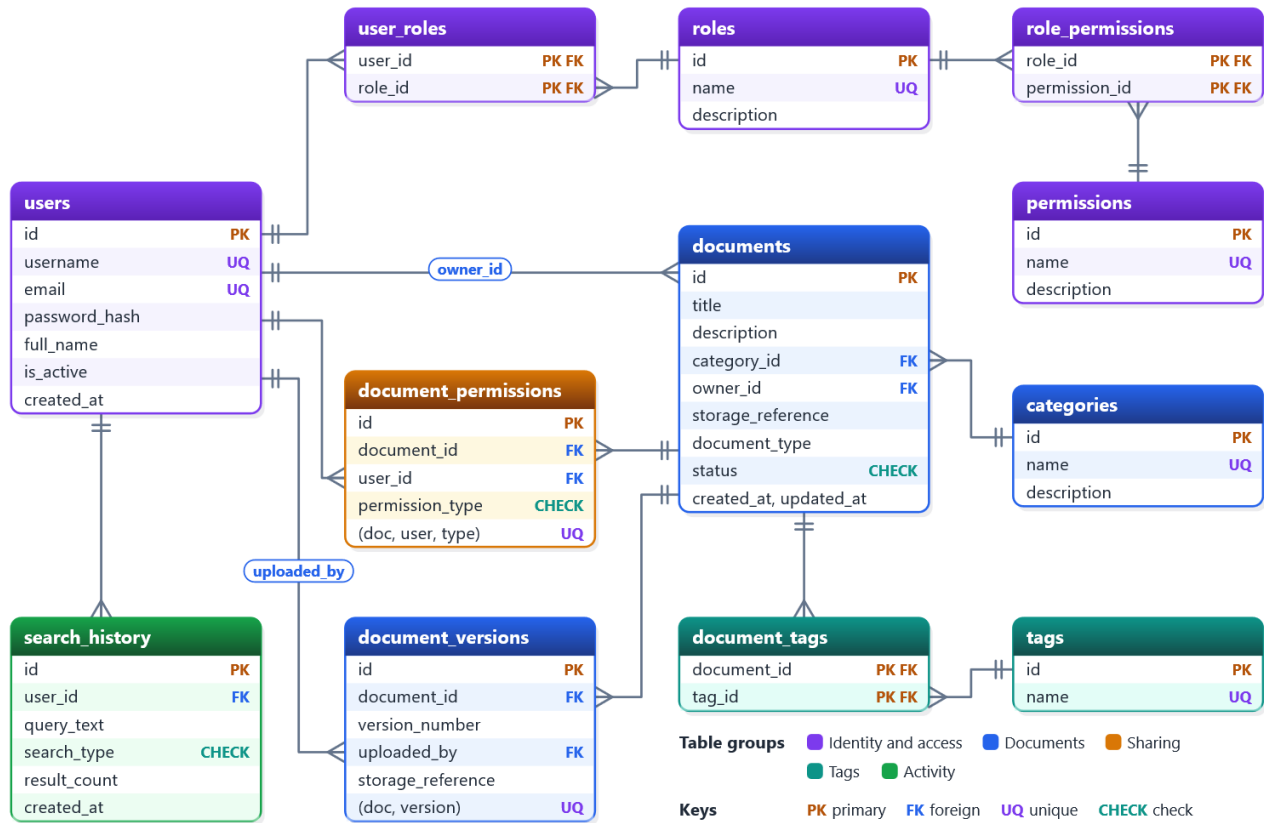


Figure 2: Entity-relationship diagram of the PostgreSQL schema (crow's-foot notation)

Table 2: PostgreSQL tables

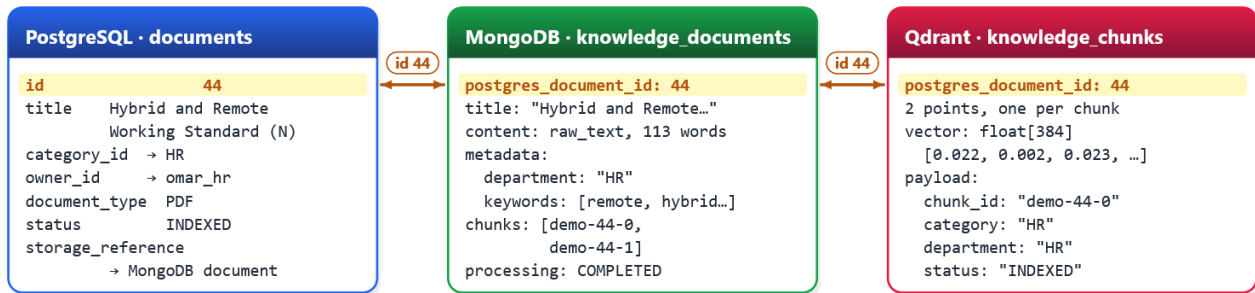
Table	Purpose	Key constraints
users	Accounts	UNIQUE username, email; BCrypt hash; is_active
roles, permissions	ADMIN, MANAGER, EMPLOYEE and permission names	UNIQUE name
user_roles, role_permissions	Many-to-many links	Composite PK; ON DELETE CASCADE
categories	Document categories	UNIQUE name
documents	Metadata and the link to the content	FKs to category and owner (SET NULL); CHECK status
document_versions	Version history	UNIQUE (document, version); CASCADE
tags, document_tags	Labels	UNIQUE name; composite PK
document_permissions	Per-user grants	CHECK type; UNIQUE (document, user, type)
search_history	Each user's searches	CHECK search_type; CASCADE on user

Normalisation and integrity. Every table is in third normal form: a document stores category_id, not the category name, and roles reach permissions through role_permissions instead of repeating them per user [3], [7]. The database enforces the rules itself: UNIQUE stops duplicate users and grants, CHECK limits statuses, grant types and search modes, and foreign-key actions decide what a deletion does (versions, tags and grants cascade with their document; a deleted owner leaves documents in place). Ten secondary indexes cover the frequent lookups, and schema changes are versioned migrations.

c. *Document Model in MongoDB and Vectors in Qdrant*

MongoDB's `knowledge_documents` collection holds one document per PostgreSQL row, linked by `postgres_document_id` [8]. A `$jsonSchema` validator requires the link, title, content, source, processing and version fields but leaves metadata open. Chunks are embedded in their document because they are always read and written with it. Nine indexes include a unique index on the link and a text index on the title and body.

Qdrant's `knowledge_chunks` collection stores one 384-dimensional all-MiniLM-L6-v2 vector [5], [6] per chunk, with cosine distance and an HNSW index ($m = 16$, $ef_construct = 100$) [4], [9]. Each point's payload carries the PostgreSQL id, chunk id, category and status. Figure 3 shows one real document in all three stores.



PostgreSQL owns identity, ownership and access; MongoDB owns the text; Qdrant owns the meaning.

Figure 3: One demo document (id 44) as stored in PostgreSQL, MongoDB and Qdrant

d. *Text Extraction for PDF, Word, Text and Markdown*

Uploads accept `.pdf`, `.docx`, `.txt` and `.md` files of up to 10 MB (Figure 4). PDFs are read with Apache PDFBox in reading order. A `.docx` file is a zip of XML parts, so the server reads `word/document.xml` with the JDK's own zip and StAX parsers, adding no dependency; DTDs and external entities are disabled, so a crafted file cannot make the server fetch or expand anything (XXE), and unzipping stops at 64 MB to defeat zip bombs. Text files must be valid UTF-8. The result is normalised and capped at 1 MB of text. Every failure returns a specific 400 or 413 message, and scanned PDFs, which hold images rather than text, are reported as needing OCR. MongoDB records which extractor produced the text (`pdfbox-3.0`, `docx-xml-1.0` or `utf8-text-1.0`).

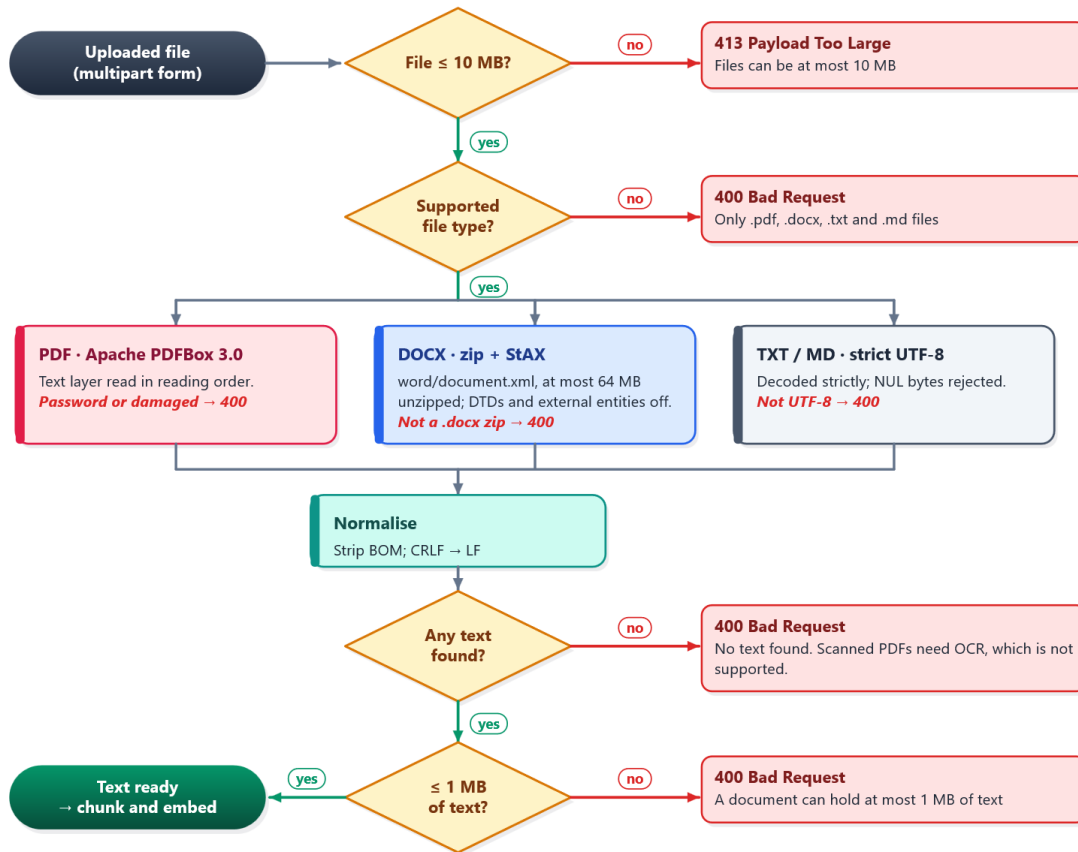


Figure 4: Text extraction: checks, one extractor per format, and the error returned at each step

e. Keeping Three Databases Consistent

No transaction spans PostgreSQL, MongoDB and Qdrant, so an upload is an ordered series of local writes, each with an undo, in the manner of sagas [10] (Figure 5). PostgreSQL is written first because it issues the id the others reference; the text is cut into 180-word chunks with a 40-word overlap and saved to MongoDB; a version row is added; chunks are embedded and upserted to Qdrant; and the status is set. A failure after step 2 removes the earlier writes in reverse order. Deletion runs Qdrant first, so an unreachable Qdrant fails the request before anything is removed. Under the CAP trade-off [11] search favours availability: without Qdrant it falls back to keyword results and says so.

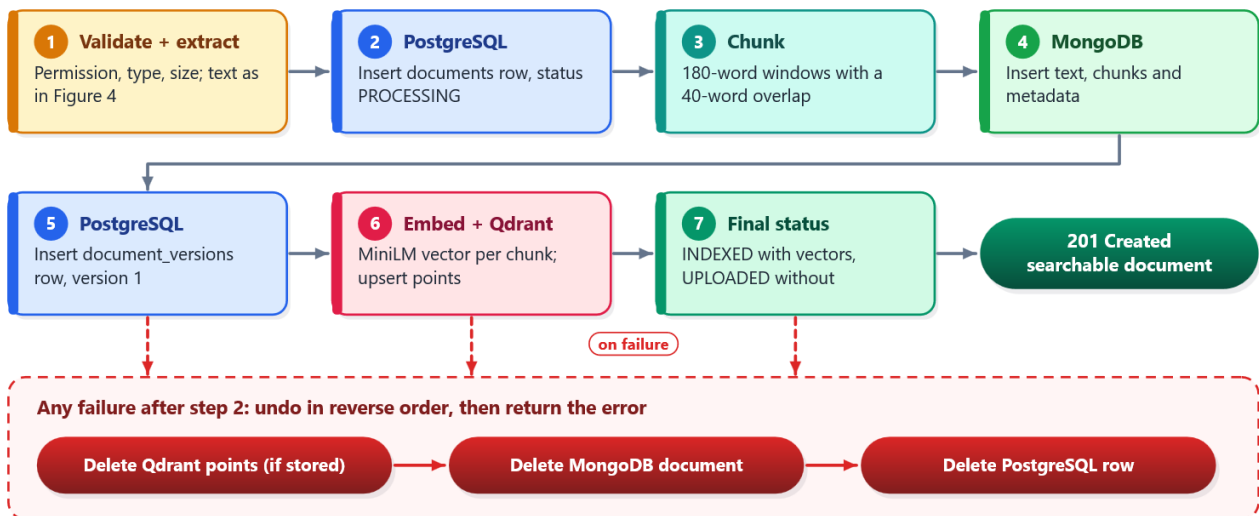


Figure 5: Upload pipeline across the three databases, with compensating actions on failure

f. *API and Security*

Controllers, services and Spring Data repositories form the backend layers [12]. Passwords are BCrypt hashes [14]; sign-in returns an HS256 JWT valid for 24 hours [13]. Figure 6 shows the checks on every request. A token is rejected as soon as its account is disabled. Endpoints require *permissions*, not role names (Table 3), and one rule in DocumentAccessService decides document reads: owner, READ grant or administrator. Search applies it *before* ranking, so even result counts never reveal hidden documents. Table 4 lists the main endpoints.

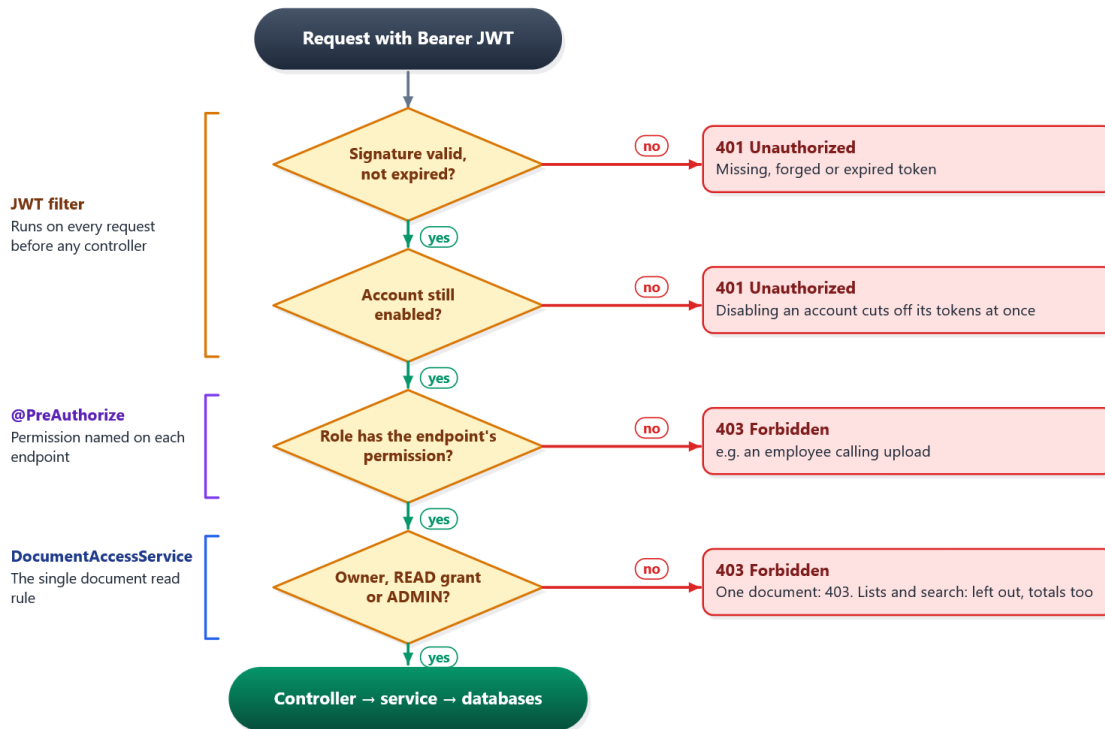


Figure 6: Authorisation checks on every request, and the response when one fails

Table 3: Roles and their permissions

Role	Permissions	Can do
ADMIN	DOCUMENT_CREATE, READ, UPDATE, DELETE, USER_MANAGE, ROLE_MANAGE	Everything; reads every document
MANAGER	DOCUMENT_CREATE, DOCUMENT_READ, DOCUMENT_UPDATE	Upload; read own and shared documents; share own
EMPLOYEE	DOCUMENT_READ	Read own and shared documents

Table 4: Main REST endpoints

Method and path	Purpose	Requires
POST /api/auth/login	Sign in, receive a JWT	Public
GET /api/documents/page	Paged, filtered readable documents	DOCUMENT_READ
GET /api/documents/{id}	PostgreSQL metadata + MongoDB text and chunks	Read access
POST /api/documents	Upload PDF, DOCX, TXT or MD (multipart)	DOCUMENT_CREATE
DELETE /api/documents/{id}	Delete from all three databases	DOCUMENT_DELETE
GET, POST, DELETE /api/documents/{id}/permissions	List, grant, revoke READ access	Owner or USER_MANAGE
POST /api/search	Unified search in four modes	Signed in; filtered
GET, POST, PATCH /api/admin/users	Manage accounts	USER_MANAGE

g. Hybrid Search

A search names a mode (hybrid, semantic, keyword or fuzzy) and optional filters (Figure 7). The **keyword leg** scores PostgreSQL candidates with the TextHack scorer from the Data Structures course: Knuth-Morris-Pratt for the whole phrase [15], Aho-Corasick for all terms in one pass [16] and Damerau-Levenshtein distance for typos [17]. The **vector leg** embeds the query with the same MiniLM model and asks Qdrant for the nearest chunks. After the permission filter, hits are merged per document and scored as $score = (0.4 \times keyword + 0.6 \times (cosine + 1) / 2) \div$ the weights of the legs that ran. Each hit reports how it matched, and every search is saved to the user's history.

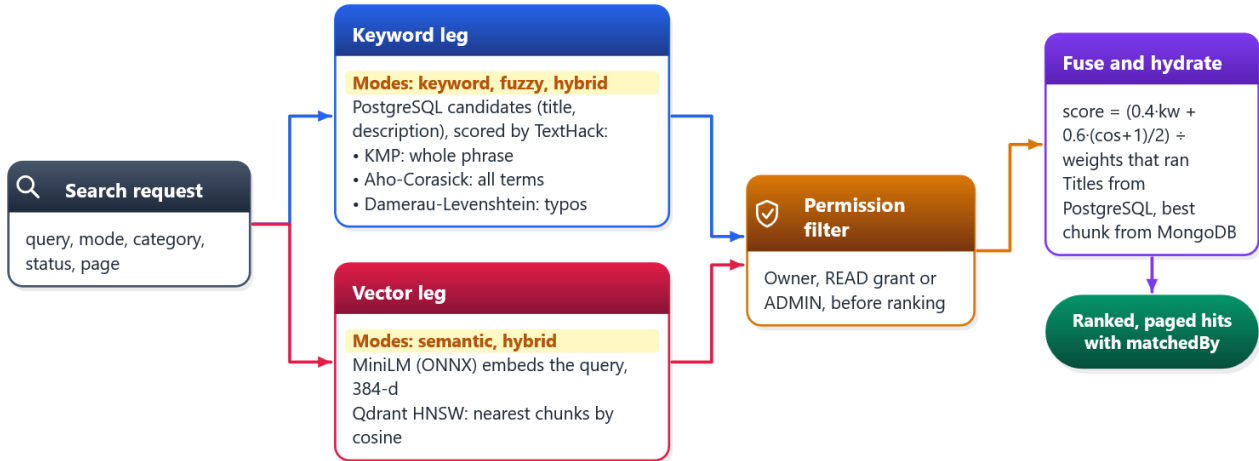


Figure 7: Hybrid search: two legs, one permission filter, weighted fusion

h. Frontend and Deployment

The React 19 frontend provides the dashboard, search, repository, document viewer, upload and administration pages, and shows only actions the user's permissions allow; the backend enforces every rule again. Both tiers ship as Docker images [18] and run locally with Compose. In production everything is on free plans: Vercel, Render (512 MB, 0.1 CPU), Neon, MongoDB Atlas and Qdrant Cloud. Fitting the backend and model into 512 MB took a 128 MB heap, one ONNX thread, 16 request threads and 4 database connections.

3. EXPERIMENTS

Integration tests run against a pinned, self-seeding stack (PostgreSQL 16.6, MongoDB 7.0.14, Qdrant 1.12.4) whose health checks wait for the seed data. A larger demo stack holds 315 documents in six categories, 705 chunk vectors, 14 users and 60 grants. The backend is tested with JUnit 5, MockMvc and Spring Security test support: unit tests use stubs (fusion, upload rollback, chunking, text extraction, the lexical scorer, embedding parity), and integration tests call the real endpoints on live databases and clean up after themselves. GitHub Actions runs the suite on every backend change. The frontend uses Vitest. On the demo stack, on 6 October 2026, we also ran:

- **Upload checks:** real files, including a 20-page PDF and DOCX produced by Word, plus damaged, password-protected, image-only and oversized files, sent over HTTP.
- **Search checks:** the same queries in different modes, including one with no shared words and one with three typos.
- **Access checks:** short-lived test tokens for eight users, recording what each could list, search and open.
- **Smoke and load tests:** 28 end-to-end checks through nginx, and 50 users for 30 s with the backend capped at 1 GB; plus a run at Render's free-tier limits.

4. RESULTS

Table 5: Test results

Suite	Result
Backend, JUnit 5 (unit + integration on live databases)	191 passed: 187 in GitHub Actions, plus 4 demo-corpus tests run locally
Frontend, Vitest (16 files)	148 passed
Smoke test through nginx	28 / 28 passed
Load test, 50 users for 30 s	169 requests/s, 0 errors

Uploads (Table 6). Real PDF and Word files were extracted, chunked, embedded and ranked first by semantic search for questions about their content. Every faulty file got the intended message, including the 413 size limit, which only a real HTTP upload can exercise.

Table 6: Upload checks with real files

File	Result
20-page report PDF made by Word, with images and tables (2.2 MB)	201, INDEXED: 5,005 words, 36 chunks, 2.1 s
The same report as Word .docx (3.1 MB)	201, INDEXED: 4,913 words, 35 chunks, 1.2 s
Course handbook PDF, 9 pages	201, INDEXED: 1,768 words, 13 chunks; ranked first (0.79) for “week-wise development schedule”
Image-only (scanned) PDF	400: no text found; scanned PDFs need OCR
Password-protected PDF	400: password-protected PDFs cannot be read
PDF renamed to .docx	400: not a readable Word (.docx) document
9 MB of random bytes named .pdf	400: not a readable PDF (passes the size limit)
File just over 10 MB	413: files can be at most 10 MB

Search (Table 7). Each leg covers what the other cannot. Keyword search alone finds nothing for “working from home rules”; hybrid search ranks the remote-working policy first through its vector leg. Fuzzy search recovers three misspelled words. One limit showed: keyword and fuzzy matching cover titles and descriptions only, so “budgt aproval” finds nothing.

Table 7: Search results on the 315-document demo corpus

Query	Mode	Hits	Top result (score, matched by)
working from home rules	Keyword	0	—
working from home rules	Hybrid	152	Hybrid and Remote Working Standard (0.42, Semantic)
what to do when the production server is down at night	Semantic	135	Incident Response and On-Call Runbook (0.63)
remte workng standrd	Fuzzy	15	Hybrid and Remote Working Standard (0.63, Keyword + Fuzzy)
budget planning	Keyword	15	Departmental Budget Planning Guidelines (1.00)
budgt aproval	Fuzzy	0	— (“approval” is in no title)

Access control (Table 8). The administrator sees all 315 documents; everyone else sees only their own and shared documents, their search totals shrink to match, and only the administrator reaches user administration.

Table 8: What each demo user can see

User	Role	Documents	Hits for “remote working”	/api/admin/users
demo_admin	ADMIN	315	137	200
priya_fin	MANAGER	48	7	403
dana_hr	MANAGER	46	31	403
mei_eng	MANAGER	31	18	403
ravi_res	MANAGER	31	7	403
marcus_legal	MANAGER	28	18	403
nils_admin	EMPLOYEE	30	15	403
grace_it	EMPLOYEE	15	15	403

Performance and defects (Tables 9 and 10). The load test exposed a lock contended by rebuilding the JWT parser per request; building it once raised throughput from 149 to 181 requests/s. At Render’s free limits the tuned image stays under 512 MB and answers semantic search in under a second.

Table 9: Performance measurements

Measurement	Result
Load test, 50 users, 30 s, backend capped at 1 GB	169 requests/s, 0 errors
Uncapped, before and after the JWT fix	149 → 181 requests/s
Semantic search at 512 MB / 0.1 CPU	0.4–0.9 s (6.5–9.4 s before tuning)
Memory after soak, concurrent searches and an upload	446 MiB steady, 485 MiB peak

Table 10: Defects found by testing, and their fixes

Defect	Fix
Three read endpoints skipped the permission rule and leaked titles and vector hits	All filter through DocumentAccessService; 7 tests
Cosine scores treated as 0–1, so fused scores could go negative	Map with $(\cos + 1) / 2$
Status filter ignored on vector hits	Filter on PostgreSQL status after merging
A disabled user’s token worked for up to 24 hours	Accept tokens only for enabled accounts
JWT parser rebuilt per request, queuing threads	Build it once at start-up

User interface. Figures 8 to 12 show the application on the demo corpus with the real backend.

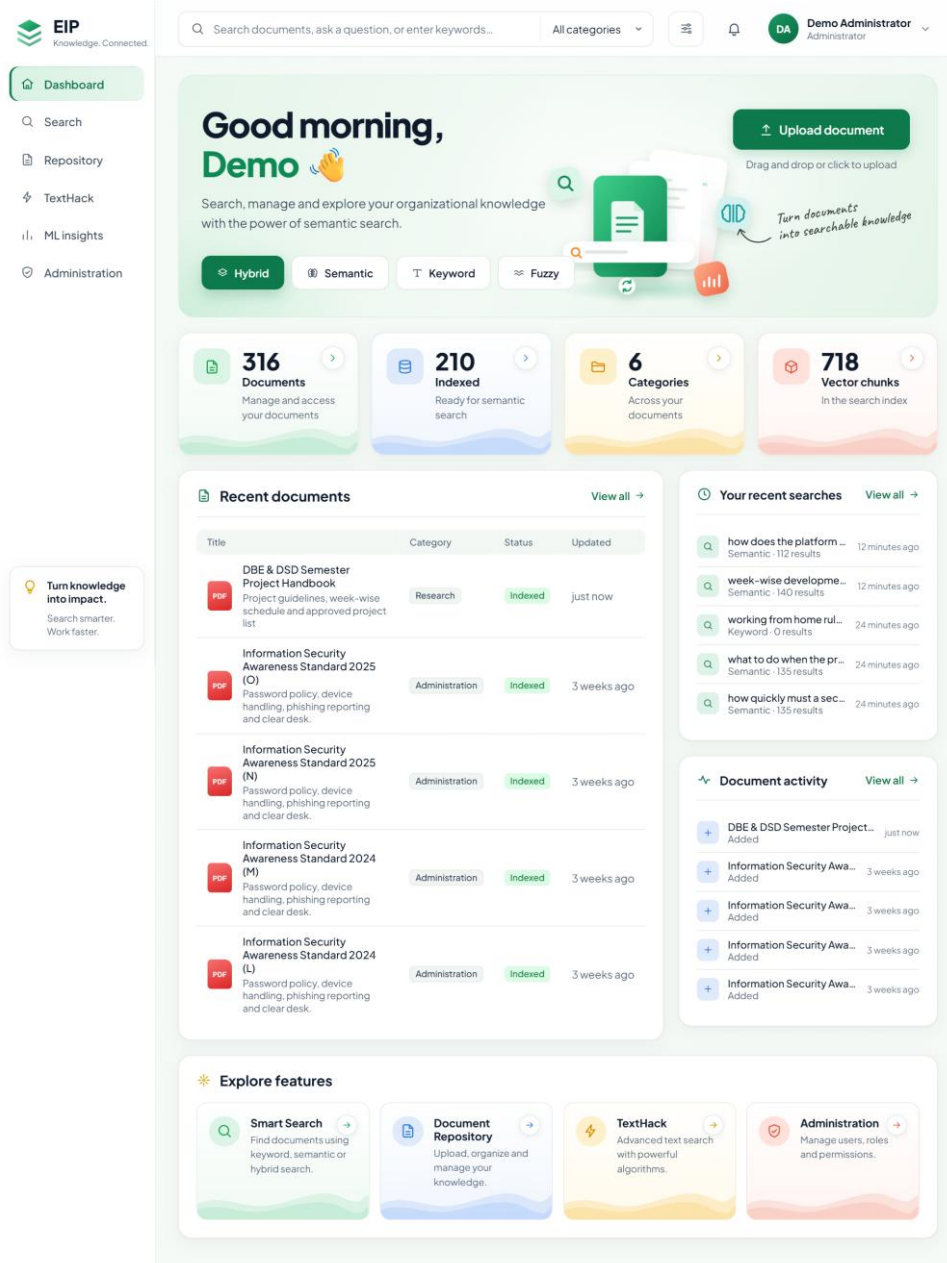


Figure 8: Dashboard with live counts from PostgreSQL and Qdrant (316 documents and 718 vectors, including the uploaded handbook)

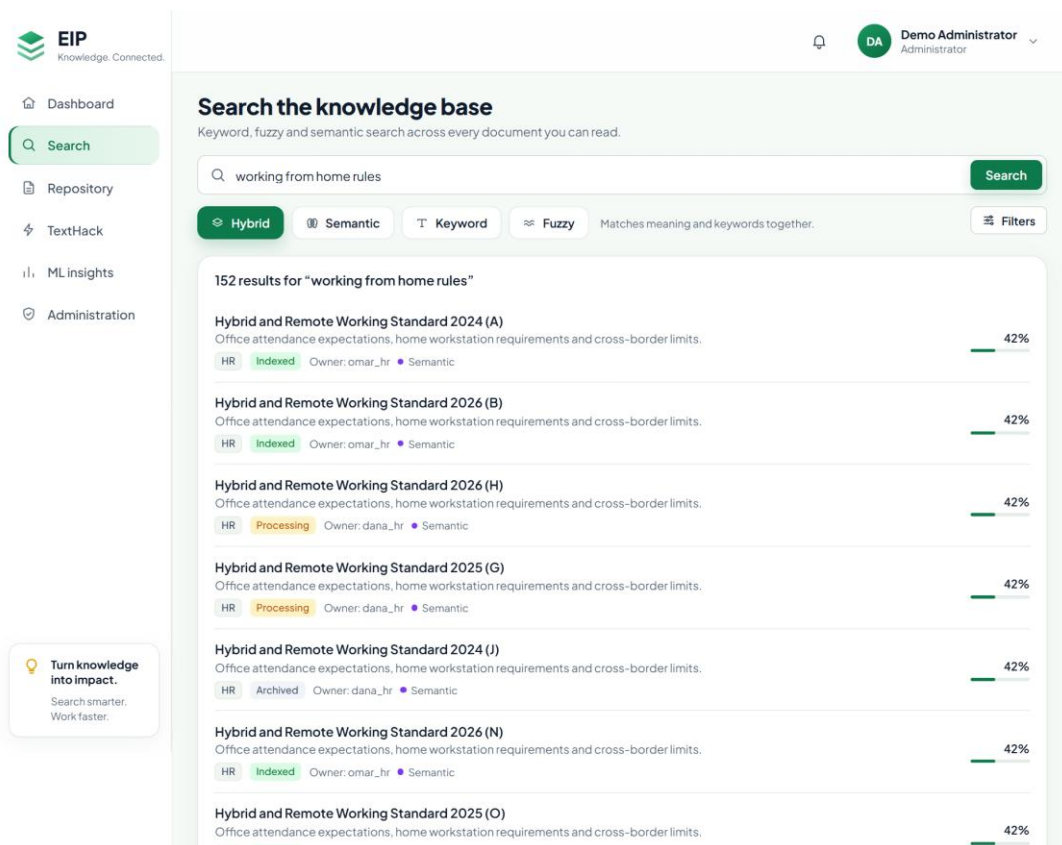


Figure 9: Hybrid search for “working from home rules”, matched by meaning

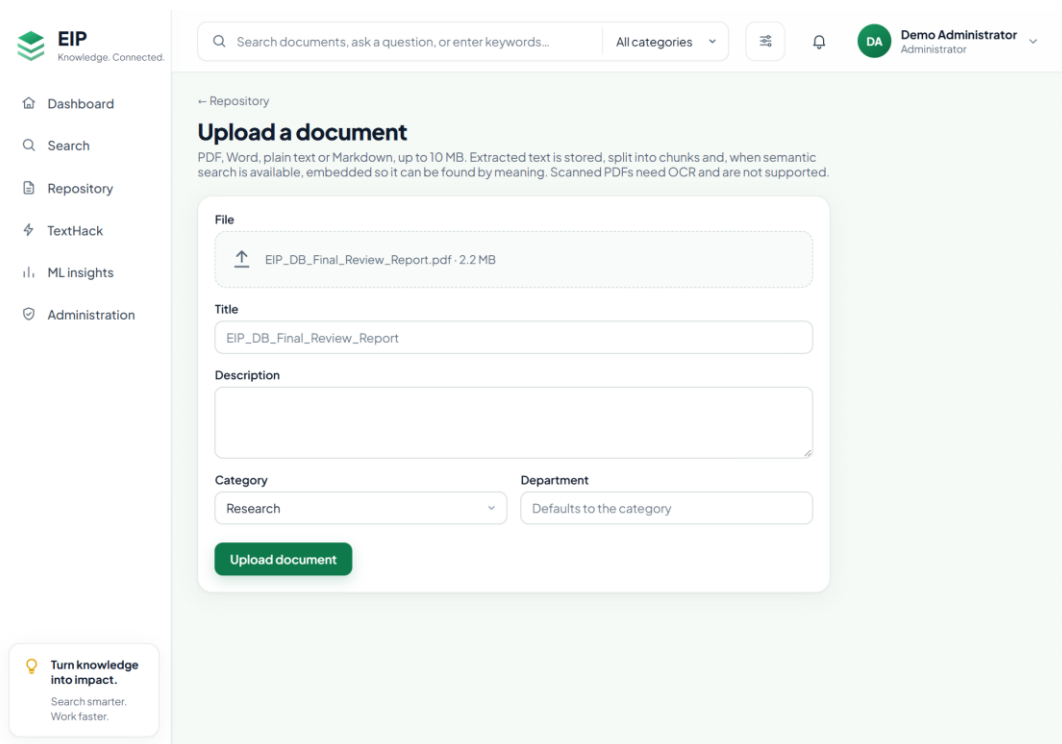


Figure 10: Upload form with a PDF chosen


EIP
 Knowledge. Connected.

Dashboard
 Search
 Repository
 TextHack
 ML insights
 Administration

Turn knowledge into impact.
 Search smarter.
 Work faster.

Search documents, ask a question, or enter keywords...
 All categories

Demo Administrator

Repository
 DBE & DSD Semester Project Handbook
 Delete

Research Indexed Owner: demo_admin Updated just now

Content
1768 words · 13569 characters

Database Systems Engineering and Distributed Backend Development Semester Project Handbook

I. Project Guidelines

- Projects shall be carried out in teams of 4 students.
- Each team shall select one approved project.
- Follow the week-wise development schedule provided by the course instructor.
- Use the prescribed technologies: PostgreSQL, MongoDB, FastAPI, Node.js, Spring Boot, Docker, and Postman.
- Follow proper database design, coding standards, and documentation throughout the project.
- Attend all weekly reviews and demonstrate the completed work.
- Maintain a Project Report documenting the design, implementation, testing, and deployment.
- Submit all deliverables (source code, database scripts, API documentation, report, and

Access

demo_admin owns this document.
 Administrators can read every document.
 No one else has been given access.

Username **Give read access**

Details

File	DEDB_Project_Handbook.pdf
Type	PDF document
Storage	MONGODB
Location	upload:DEDB_Project_Handbook.pdf
Processing	Completed
Processed	Oct 6, 2026, 3:11 AM
Extractor	pdfbox-3.0
Chunker	words-180-40
Version	1
Change	Initial upload
Created	Oct 6, 2026, 3:11 AM

Metadata

Department	Research
Uploaded by	demo_admin

Chunks 13

#1 doc338-chunk1
 Database Systems Engineering and Distributed Backend Development Semester Project Handbook I. Project Guidelines • Projects shall be carried out in teams of 4 students. • Each team shall select one approved project. • Follow the week-wise development schedule provided by the course instructor. • Use the prescribed technologies: PostgreSQL, MongoDB, FastAPI, Node.js, Spring Boot, Docker, and Postman. • Follow proper database design, coding standards, and documentation throughout the project. • Attend all weekly reviews and demonstrate the completed work. • Maintain a Project Report documenting the design, implementation, testing, and deployment. • Submit all deliverables (source code, database scripts, API documentation, report, and presentation) before the specified deadlines. • Ensure that the project work is original and adheres to academic integrity guidelines. • Every team member

Figure 11: An uploaded PDF in the viewer: MongoDB text and chunks, extractor pdfbox-3.0

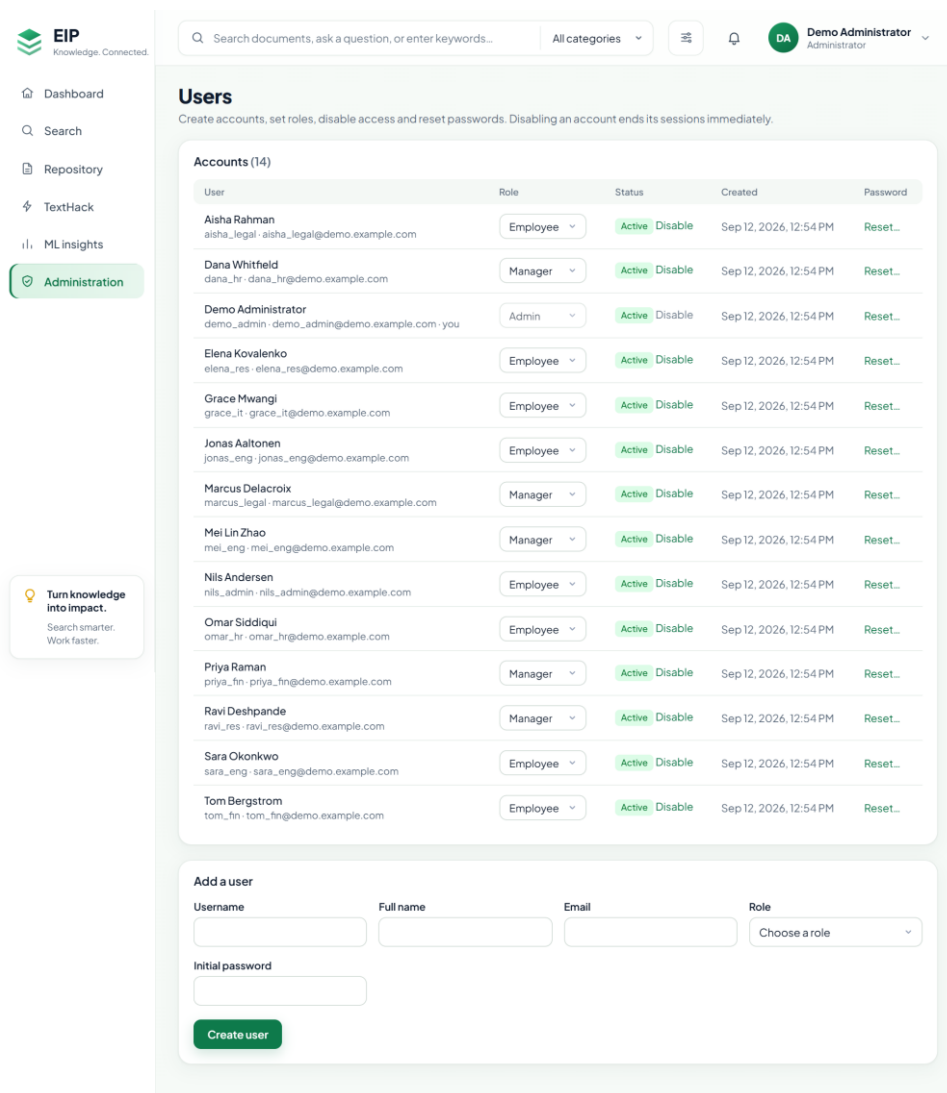


Figure 12: User administration (USER_MANAGE only)

5. CONCLUSION AND FUTURE WORK

Relational, document and vector databases work as one system when each owns a clear part of the data and one key links them. PostgreSQL enforces identity, access and integrity; MongoDB keeps text and chunks in a validated, flexible model; Qdrant makes them searchable by meaning. Safe text extraction brings PDF and Word documents in, ordered writes with compensating actions keep the stores consistent, and one permission rule applied before ranking keeps every result within what the user may read. The tests, the measurements on the demo corpus and the live deployment confirm the design.

Future work.

- OCR for scanned PDFs, and text from Word headers, footers and footnotes.
- Keyword and fuzzy matching over document text, using the MongoDB text index.
- Readable-document filters passed to Qdrant, so restricted users always get a full page of vector hits.
- Background embedding for large uploads.
- Separate search and ingestion services behind an API gateway, with Prometheus and Grafana monitoring [19].

REFERENCES

- [1] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley, 2012.

- [2] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.
- [3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*. McGraw-Hill Education, 2018.
- [4] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [6] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] L. Ferrari and E. Pirozzi, *Learn PostgreSQL*, 2nd ed. Packt Publishing, 2024.
- [8] A. Giamas, *Mastering MongoDB 4.x*, 2nd ed. Packt Publishing, 2019.
- [9] Qdrant, "Qdrant documentation: collections, indexing and payload filtering." [Online]. Available: <https://qdrant.tech/documentation/>
- [10] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.
- [11] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
- [12] C. Walls, *Spring in Action*, 6th ed. Manning Publications, 2022.
- [13] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.
- [14] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 1999.
- [15] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [16] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [17] F. J. Damerau, "A technique for computer detection and correction of spelling errors," *Communications of the ACM*, vol. 7, no. 3, pp. 171–176, 1964.
- [18] N. Poulton, *Docker Deep Dive*. Independently published, 2023.
- [19] M. Larsson, *Microservices with Spring Boot 3 and Spring Cloud*. Packt Publishing, 2023.